Eötvös Loránd Tudományegyetem

Informatikai Kar

Programozáselmélet és Szoftvertechnológiai Tanszék

Keresőalgoritmusok interaktív alkalmazása és állapotgráfok ábrázolása

Témavezető:
Dr. Gregorics Tibor
Tanszékvezető egyetemi docens

Szerző:
Szécsi Szabolcs Ádám
Programtervező informatikus BSc

Budapest, 2026

Tartalomjegyzék

1. Bevezetés **3**

2. Felhasználói dokumentáció **5**

2.1. A probléma meghatározása 5

2.2. A módszerek áttekintése 6

2.3. Rendszerkövetelmények 7

2.4. Az alkalmazás áttekintése 8

2.5. Szint betöltése 10

2.6. Kézi játék 10

2.7. A keresőalgoritmus konfigurálása 11

2.8. Keresőalgoritmusok futtatása 12

2.9. Állapotok felfedezése 13

2.10. Interakció a 3D-s állapotgráffal 13

2.11. Színmagyarázat 14

3. Fejlesztői dokumentáció **16**

3.1. Elemzés 16

3.1.1. Feladatleírás 16

3.1.2. Funkcionális leírás 16

3.1.3. Nem-funkcionális követelmények 20

3.2. Tervezés 20

3.2.1. Szoftverarchitektúra 20

3.2.2. Modell 21

3.2.3. Nézet 23

3.2.4. Adattárolás 25

3.2.5. Keresőalgoritmusok és heurisztikák 26

3.3. Implementáció 30

3.3.1. Felhasznált technológiák 30

3.3.2. Eltérések az eredeti tervtől 30
3.4. Tesztelés 31
3.4.1. Egységtesztek 31
3.4.2. Végfelhasználói tesztesetek 33

4. Összegzés **36**

Irodalomjegyzék **37**

Ábrajegyzék **38**

Táblázatjegyzék **39**

1. fejezet

Bevezetés

A keresőalgoritmusok meghatározó szerepet játszanak az informatikában és azon belül a mesterséges intelligencia területén. Ezek az algoritmusok elengedhetetlenek az útkeresési feladatokban, a játékok mint a sakk elemzésében, az automatikus következtetésben is. Bár ezeknek az algoritmusoknak az elméleti tulajdonságai, mint például a teljesség, optimalitás és a számítási komplexitás már jól ismertek, maguk az algoritmusoknak a megértésének elsajátításához nem találunk sok eszközt. Azonban a felfedezett állapottér ábrázolásával sokkal világosabbá válhat, hogy adott kereső algoritmusok, milyen szabályok szerint hoznak döntéseket, a globális munkaterét ábrázolva, megfigyelhető, hogy milyen megszerzett tudással tud dolgozni az algoritmus. Kielemezhető, hogy milyen szerepet játszanak a jó heurisztikus függvények, és ez által milyen eredményeket produkálnak különböző problémákon.

Ez a dolgozat egy olyan interaktív alkalmazást mutat be, amely keresőalgoritmusokat alkalmaz a Rush Hour logikai játékra [1], és a feltárt állapotgráfot háromdimenziós térben vizualizálja. A Rush Hour egy olyan rejtvény, amelyben a játékosnak egy 6×6-os táblán kell járműveket csúsztatnia, hogy utat törjön a fő (piros) autónak a kijárat felé. A játék szabályai nagyon egyszerűek, viszont a feladványok komplexitása a viszonylag könnyűtől a meglepően bonyolult nehézségig terjed, ezért ideális választásnak bizonyult az elemzésre kerülő problémaként.

A projekt két oldalról közelíti meg a problémát:

1. **Keresőalgoritmusok implementálása és vezérlése.** Három keresőalgoritmus került implementálásra: a hegymászó keresés általános verziója - a tabu keresés, visszalépéses (backtracking) keresés és A^C (A és A^* speciális változata) gráfkeresés, amelyek mindegyike konfigurálható a négy heurisztikus függvény

egyikével és egy opcionális véletlenszerűségi beállítással. A felhasználó manuálisan is játszhat a rejtvénnyel, és minden felfedezett állapot megjelenik az állapotgráfban és minden szomszédos állapot össze lesz kötve.

2. **Háromdimenziós állapotgráf-vizualizáció.** A felfedezett állapotok kör alakú csúcsként jelennek meg a három dimenziós térben, amelyeket az érvényes lépéseket reprezentáló élek kötnek össze. Egy erőalapú elrendezési (force-directed layout) fizikai szimuláció mozgatja a csúcsokat úgy, hogy a szomszédos állapotok viszonylag közel maradjanak egymáshoz, míg a nem szomszédosak távolodjanak, ehhez az élek mentén rugóerőket, a csúcspárok között pedig a távolság négyzetének reciprokával arányos taszítóerőt alkalmazunk. A taszítási számítások $O(V^2)$ költségének kezelése érdekében a Barnes–Hut optimalizációs algoritmust [2] és egy nyolcasfa (octree) térbeli adatszerkezetet használunk, ami az időbeli komplexitást képkockánként $O(V \log V)$-re csökkenti. (V a felfedezett állapotok azaz a megjelenített csúcsok száma)

Az alkalmazás Godot Engine 4.6 [3] használatával, C# és .NET [4] nyelven készült, és a Modell–Nézet (Model–View) arkitektúrát alkalmazza. A modell réteg tiszta C# konstrukciókkal enkapszulálja a rejtvény logikáját, a keresőalgoritmusokat és a heurisztikus függvényeket, míg a nézet réteg a Godot jelenetrendszerét használja a megjelenítéshez, a fizikai szimulációhoz és a felhasználói interakcióhoz.

2. fejezet

Felhasználói dokumentáció

Ez a fejezet az alkalmazás teljes leírását nyújtja a felhasználó számára. A probléma összefoglalásával kezdődik, ismerteti az alkalmazott módszereket, majd útmutatást ad a program futtatásához és a vele való interakcióhoz.

2.1. A probléma meghatározása

A Rush Hour rejtvény [1] egy tologatós játék, amelyet egy 6×6-os négyzetrácsos táblán játszanak. A tábla több féle járművet tartalmazhat: 1×2 mezőt elfoglaló **autókat** és 1×3 mezőt elfoglaló **buszokat**. Minden jármű csak a előre és hátra a hosszabbik tengelye mentén mozoghat, és nem tudják egymást odébb tolni. A cél a kijelölt (piros színű) **főautó** vízszintes elcsúsztatása, amíg az el nem éri a tábla jobb szélén található kijáratot (3. sor, a tábla jobb oldalán).

A rejtvény akkor tekinthető **megoldottnak**, ha a főautó orra közvetlenül a kijáratnál helyezkedik el. A játék állapottere az összes a kezdőállapotból elérhető konfigurációból áll, az állapotok közötti átmenetek pedig egy-egy jármű egy mezővel való elmozdításának felelnek meg a megengedett irányban.

Az alkalmazás két egymáshoz kapcsolódó kihívásra fókuszál:

1. Keresőalgoritmusok implementálása és interaktív vezérlése a Rush Hour rejtvényen, megengedve a felhasználónak hogy tetszőlegesen megállítsa, léptesse, folytassa az algoritmus futtatását, és közben minden döntést ki tudjon elemezni.

2. Az állapotgráf vizualizációja, ahol a csúcsok a felfedezett állapotok, az élek pedig a megengedett lépések egy háromdimenziós színtérben, erőalapú elren-

dezéssel (force-directed layout) szimulálva, hogy a szerkezetileg összefüggő állapotok közel kerüljenek egymáshoz.

2.2. A módszerek áttekintése

Az alkalmazás három keresőalgoritmust valósít meg, amelyek mindegyike heurisztikus vezérlési startégiát alkalmaz konfigurálható heurisztikus függvényekkel:

Tabu keresés Egy nemmódosítható lokális keresési módszer, amely egy heurisztika alapján választja ki a legjobb szomszédos állapotot, miközben memóriát tart fenn a legutóbb meglátogatott állapotokról (*tabu lista*), hogy elkerülje a köröket.

Visszalépéses (Backtracking) keresés Módosítható stratégiájú keresés, amely heurisztika által vezérelt sorrendben tárja fel az állapotokat a köröket elkerülve, zsákutca esetén visszalép, véges állapotgráfra garantálja a leállást.

$\mathbf{A^C}$ gráfkeresés Módosítható statégiájú gráfkeresés, amely az $f(n) = g(n) + h(n)$ kiértékelő függvényt használja, ahol $g(n)$ a mélység (a kezdőállapottól megtett lépések száma), $h(n)$ pedig a heurisztikus érték. Megengedhető és monoton heurisztikákkal ez az algoritmus megtalálja az optimális megoldást.

Négy heurisztikus függvény áll rendelkezésre, amelyek mindegyike nemnegatív, megengedhető és monoton:

- **Null heurisztika** ($h = 0$): nem ad útmutatást; a „brute-force” viselkedés megfigyelésére hasznos.
- **Távolság heurisztika**: a kijuttatandó autó távolsága a kijárattól (tartománya: 0–4).
- **Távolság + blokkolás heurisztika**: a távolság, plusz egy költség minden egyes, a főautó és a kijárat között lévő foglalt mezőért.
- **Legkevesebb lépés heurisztika**: Egy mégerősebb alsó becslést ad a megoldásig szükséges lépésekre: a távolság, plusz minden a fúautó útjába álló járművek elmozdításához szükséges minimális lépésszám, hogy már ne legyen útban.

A 3D-s állapotgráf elrendezéséhez erőalapú megközelítést alkalmazunk: az élek rugókként működnek, amelyek adott távolság felé húzzák a szomszédos állapotokat, míg minden csúcspár egymásra a távolságuk négyzetével fordítottan arányos

taszítóerőt fejtenek ki egymásra. A számításigényes $O(V^2)$ komplexitású taszítást a Barnes–Hut algoritmus [2][5] nyolcasfa (octree) térbeli adatszerkezet implementálásával approximáljuk, és ezzel az idő komplexitását le tudjuk csökkenteni $O(V \log V)$-re.

2.3. Rendszerkövetelmények

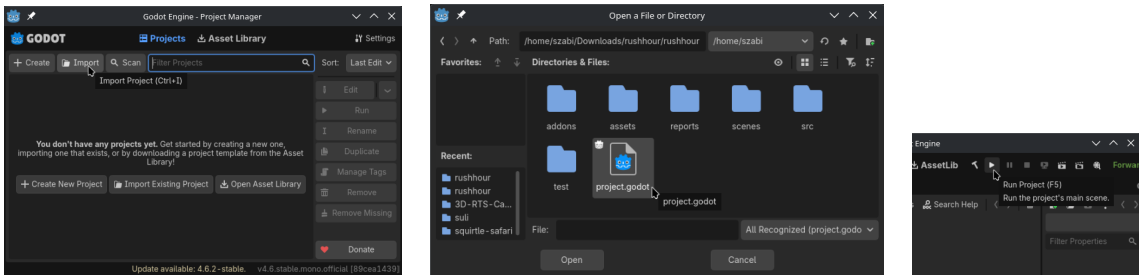
Az alkalmazás önállóan futtatható fájlként érhető el windowson és linuxon, és nem igényel további szoftvertelepítést. Van lehetőség a projektet a godot editorból futtatni és exportálni futtatható fájl formátumba egyéb platformokra. Ajánlott minimum képernyőfelbontás: 1280×720

Platform	**Futtatás**
Windows	Futtassa közvetlenül a mellékelt `rushhour.exe` fájlt.
Linux	Futtassa közvetlenül a mellékelt `rushhour.x86_64` fájlt.
macOS / egyéb	Töltse le és csomagolja ki a `rushhour.zip` fájlt és nyissa meg (Import project) a kicsomagolt `project.godot` fájlt a Godot 4.6 Mono szerkesztővel, majd jobb felüli gombbal futtassa.

2.1. táblázat. Támogatott platformok és futtatási utasításaik

A projekt szerkesztőben való megnyitásához szükséges előfeltételek:

- Godot 4.6 Mono (C#) szerkesztő le van töltve. [3]
- .NET SDK 9.0 vagy újabb verzió telepítve van. [4]

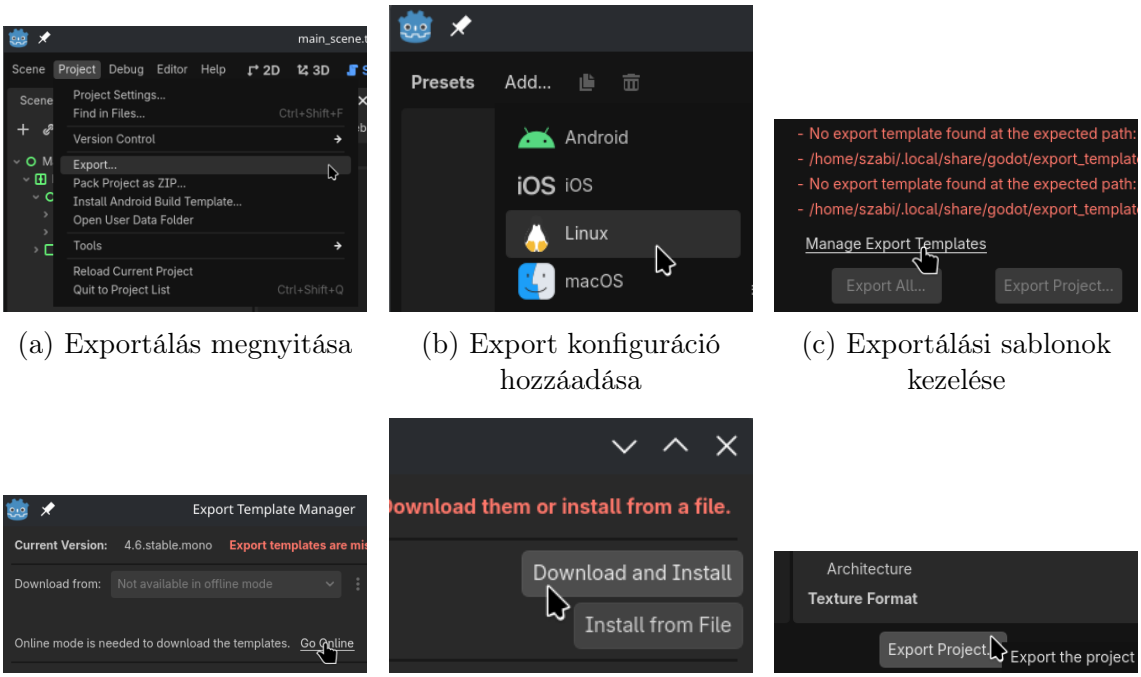


(a) Projekt importálása (b) Projekt megnyitása (c) Projekt futtatása

2.1. ábra. A projekt megnyitása és futtatása a Godot szerkesztőből

Az forrásból való fordításhoz, ha az előfeltételek adottak, a projekt megnyitása után bal felül Project > Export opciónál található, hozzá kell adni egy exportálási

konfigurációt a kiválasztott platformra, majd le kell tölteni még egy exportálási könyvtárat amit fel fog ajánlani a szerkesztő.



(a) Exportálás megnyitása (b) Export konfiguráció hozzáadása (c) Exportálási sablonok kezelése

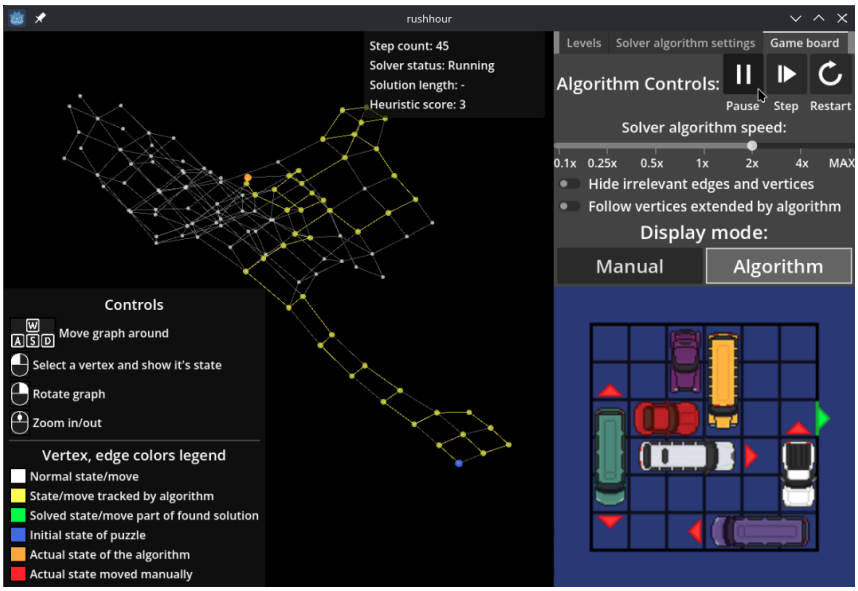
(d) Online módba lépés (e) Export sablonok letöltése (f) Projekt exportálása

2.2. ábra. A projekt exportálásának lépései a Godot szerkesztőből

2.4. Az alkalmazás áttekintése

Az alkalmazás ablaka két fő területre oszlik:

- **Bal oldali panel**: a 3D-s állapotgráf vizualizációs színtere.
- **Jobb oldali panel**: Az konfigurációs és vezérési opciók 3 fülre osztva



2.3. ábra. Az alkalmazás egy algoritmus futása közben

A Bal oldali panelen belül található:

- Bal alul az állapotgráffal való interakciónak az irányításai
- Jobb felül a jelenleg futó algoritmushoz kapcsolódó információk:
  - Az algoritmus jelenlegi státusza
  - Az algoritmus által megtett lépések száma
  - A megtalált megoldás lépésszáma
  - A jelenleg táblán mutatott állás heurisztikus értéke

A Jobb oldali panel fülekre osztva:

- **Feladvány választó fül**: Kattintással lehet választani az előnézettel felsorolt feladványok közül.
- **Keresőalgoritmus beállítások fül**: Indítható az állapotok felfedezése, beállítható, hogy milyen keresőalgoritmust, milyen heurisztikát szeretnénk alkalmazni a kiválasztott feladványra.
- **Játék tábla fül**: Tartalmazza a játék tábláját és a manuális és algoritmus mód választó gombokat, az algoritmust vezérlő: megállítás, folytatás, léptetés, újraindítás gombokat, egy csúszkát a az algoritmus léptetési sebességéhez és kapcsolókat az állapotgráf megjelenéséhez kapcsolódóan.

2.5. Szint betöltése

Az alkalmazás különböző nehézségű, kiválogatott Rush Hour rejtvények gyűjteményét tartalmazza. Az összes rejtvény görgetve tekinthető meg.

Bármikor másik feladvány választható az alábbi módon:

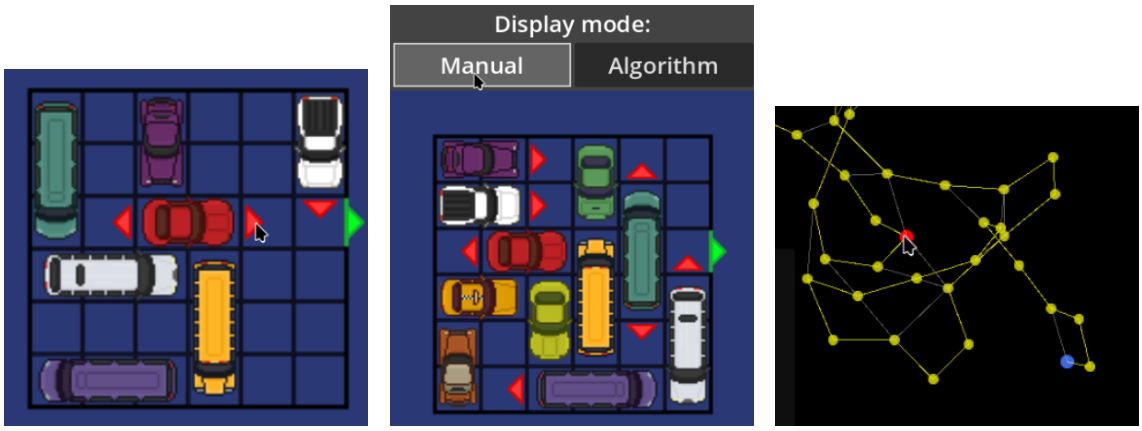
1. Navigáljon a jobb felül a füleken belül a **Levels** fülre.
2. Kattintson a kívánt rejtvénynek megfelelő gombra.
3. A játéktábla betölti a kezdeti konfigurációt, a 3D-s állapotgráf térben pedig megjelenik az a csúcs, amely a kezdőállapotot reprezentálja.

2.6. Kézi játék

A rejtvény manuálisan is játszható a játéktáblával való közvetlen interakció útján:

1. Minden mozgatható jármű mellett piros nyíl indikátorok jelennek meg, mutatva a lehetséges mozgási irányokat.
2. Kattintson egy nyílra a megfelelő jármű egy mezővel való eltolásához.
3. Minden kézi lépés egy új állapotot, valamint egy hozzá tartozó csúcsot és élet hoz létre a 3D-s gráfon, lehetővé téve a felhasználó számára az állapotgráf interaktív felépítését.

A manuális módba bármikor át lehet kapcsolni a **Manual** gombra nyomással, vagy egy már felfedezett állapotnak a megjelenített csúcsára kattintani, és ekkor a tábla a kiválasztott állapotot megjeleníti és manuális módban lesz a tábla, mindkét esetben, ha ekkor épp fut kereső algoritmus akkor az a háttérben fog tovább futni.



(a) A feladvány megoldása manuális lépésekkel keresőalgoritmus alkalmazása nélkül

(b) A tábla már nem követi az algoritmus lépéseit manuális módba váltás után.

(c) Állapot kiválasztása egy felfedezett csúcsra kattintás esetén

2.4. ábra. Manuális játék és csúcsra kattintás

2.7. A keresőalgoritmus konfigurálása

A keresőalgoritmus futtatása előtt a felhasználó a **Solver settings** fülön konfigurálhatja a paramétereket:

Algoritmus választása Válasszon a Tabu keresés, Visszalépéses keresés vagy A^C gráfkeresés közül.

Tabu méret (Csak Tabu keresésnél) Egy $n \geq 0$ egész szám, amely meghatározza, hány legutóbb meglátogatott állapotra emlékezzen a rendszer.

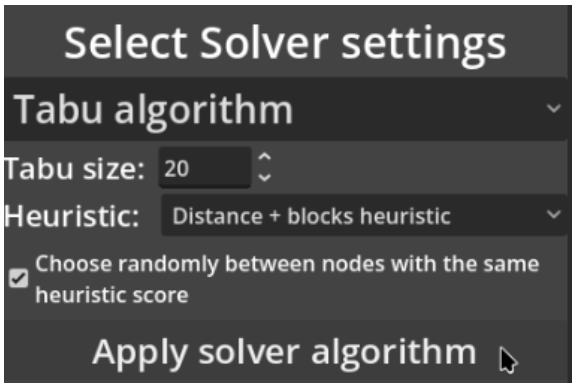
- $n = 0$: mohó algoritmus — memória nélkül mindig a legjobb szomszédra lép.
- $n = 1$: hegymászó algoritmus — csak az előző állapotra emlékszik, és elkerüli az oda való visszalépést.
- $n > 1$: elkerüli az utolsó n meglátogatott állapotot. Ha minden lehetséges lépés tabu állapotba vezetne, az algoritmus megoldás nélkül terminál.

Heurisztikus függvény Válasszon a négy elérhető heurisztika közül (Null, Distance, Distance + blocks, Least-moves).

Véletlenszerűség kapcsoló Ha engedélyezve van, az azonos heurisztikus pontszámú állapotok közül az algoritmus véletlenszerűen választ a determinisztikus

választás helyett, ami ismételt futtatásoknál különböző bejárási utakat eredményezhet.

Alkalmaz (Apply) gomb Nyomja meg, hogy a beállítások szerinti keresőalgoritmus inicializálva legyen a betöltött feladványra.



2.5. ábra. Keresőalgoritmusok beállítása és alkalmazása

2.8. Keresőalgoritmusok futtatása

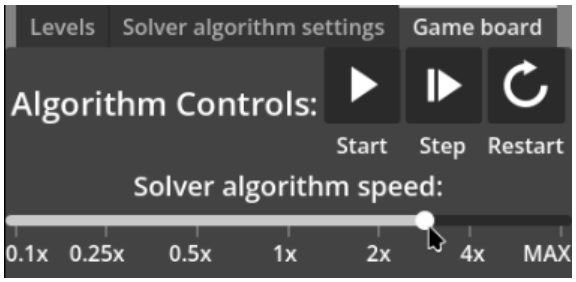
A szint betöltése és a megoldó konfigurálás és alkalmazás (Apply gomb) után az algoritmus az **Game board** fülről vezérelhető:

Lejátszás/Szünet Az algoritmus folyamatos futtatásának indítása vagy megállítása.

Léptetés (Step) Az algoritmus egyetlen lépésének végrehajtása, ami hasznos a részletes megfigyeléshez.

Újraindítás Az algoritmus visszaállítása a kezdőállapotba a jelenlegi beállítások megtartásával.

Sebességcsúszka Az algoritmus lépései közötti késleltetés beállítása. A csúszka jobbra tolásával a sebesség nő; a maximum állásban az algoritmus a lehető leggyorsabban fut, képkockánként több szász lépést is végrehajthat.



2.6. ábra. Algoritmus lépéseinek vezérlése

A játéktábla alatti állapotsor a következőket jeleníti meg:

- Az algoritmus által végrehajtott aktuális **lépésszám** (step count).
- A **megoldó állapota** (Inicializálva, Fut, Megoldva, Nincs megoldás, Zsákutcába jutott).
- A **megoldás hossza** (lépések száma), ha talált az algoritmus megoldást.
- Az éppen megjelenített állapot **heurisztikus értéke**.

Amikor az algoritmus megoldást talál, a megoldási útvonal zölddel kiemelve jelenik meg az állapotgráfon, és láthatóvá válik a megoldás hossza.

2.9. Állapotok felfedezése

A keresőalgoritmusokon kívül az alkalmazás **BFS** (szélességi keresés) és **DFS** (mélységi keresés) felfedező módokat is kínál, amelyek a Solver settings fülről érhetők el. Ezek a módok anélkül tárják fel az állapotteret, hogy megoldást keresnének, minden elérhető állapotot és élet hozzáadva az állapotgráfhoz:

- Beállítható egy **maximális állapotszám** a felfedezett állapotok korlátozására. Ez ajánlott a nehezebb feladványoknál, ahol az állapottér több ezer elemet is tartalmazhat, mivel a túl sok csúcs átláthatatlanná teheti az állapotgráfot, lelassíthatja a fizikai szimulációt.
- Az **unlimited** kapcsoló eltávolítja az állapotlimitet.

A felfedezés nagy sebességgel fut és feltölti a gráfot, ami után a felhasználó futtathat egy keresőalgoritmust a már teljesen (vagy részben) feltérképezett állapottéren.

2.10. Interakció a 3D-s állapotgráffal

Az alkalmazás bal oldalán található 3D-s térben jeleníti meg az állapotgráfot. Az interakció a következőképpen történik:

Kamera forgatása Tartsa lenyomva a jobb egérgombot és húzza az egeret a gráf körüli keringéshez.

Kamera mozgatása Használja a `W`, `A`, `S`, `D` billentyűket a kamera aktuális irányához képesti eltolásához. Shift nyomva tartásával a mozgás gyorsabb.

Zoom Használja az egérgörgőt a közelítéshez és távolításhoz.

Csúcsra kattintás Kattintson a bal egérgombbal egy csúcsra az adott állapotba ugráshoz: a játéktábla Kézi módba vált és a kiválasztott állapotot mutatja.

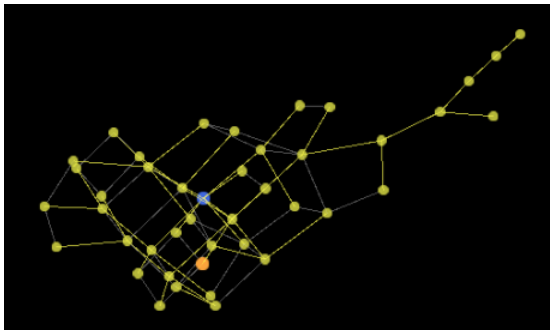
Csúcsok/élek elrejtése Egy kapcsológombbal minden csúcs és él szinte teljesen átlátszóvá tehető. Ez hasznos lehet a vizuális zaj csökkentésére nagy gráfok felfedezésekor.

2.11. Színmagyarázat

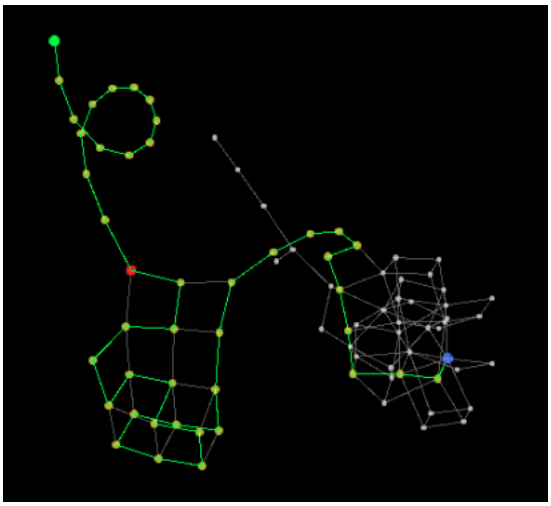
A 3D-s gráf csúcsai és élei színkódoltak a szerepük szemléltetésére:

Elem	**Szín**	**Jelentés**
Csúcs	Piros	Manuálisan kiválasztott aktuális állapot
Csúcs	Zöld	Megoldott állapot (főautó a kijáratnál)
Csúcs	Kék	Az aktuális feladvány kezdőállapota
Csúcs	Narancs	Az algoritmus aktuális állapota
Csúcs	Sárga	Az algoritmus memóriájában tárolt állapot
Csúcs	Fehér	Felfedezett állapot
Él	Zöld	A megtalált megoldási útvonal része
Él	Sárga (áttetsző)	Az algoritmus memóriájában tárolt lépés
Él	Fehér (halvány)	Felfedezett lépés

2.2. táblázat. A 3D-s állapotgráf csúcsainak és éleinek színmagyarázata



(a) Gráfkereső algoritmus memóriájában tárolt feszítőfa látszik sárga színnel, az aktuális állapot narancssárgán van ábrázolva.



(b) Megoldott állapot zölddel kitüntetve, hozzá vezető zöld élek mentén a megoldás, a megoldási úton egy állapot van kiválasztva piros színnel megjelenítve.

2.7. ábra. Állapotgráf színkódolás szemléltetése

Egy állapot csúcsára egyszerre több tulajdonság is vonatkozhat, például épp kiválaszott és algoritmus memóriájában tárolt állapot, ekkor a táblázatban felsorolt sorrend szerint a legelső tulajdonságnak a megjelenítése élvez prioritást. Az első négy tulajdonság (kézi aktuális, megoldott, kezdő, algoritmus aktuális), nem csak színben másak, de kétszeres méretben is nagyobbként jelennek meg. Az algoritmus memóriájában tárolt csúcsok másfélszeres méretben vannak megjelenítve.

3. fejezet

Fejlesztői dokumentáció

Ez a fejezet az alkalmazás elemzését, tervezését, implementációját és tesztelését mutatja be.

3.1. Elemzés

3.1.1. Feladatleírás

A feladat egy olyan interaktív alkalmazás fejlesztése, amely:

1. Modellezi a Rush Hour tologatós rejtvényt [1] egy immutábilis állapot-átmenet rendszerként egy 6×6-os táblán, 1×2-es autókkal és 1×3-as buszokkal.
2. Implementál három konfigurálható keresőalgoritmust (tabu keresés, visszalépéses keresés, A^C gráfkeresés) négy heurisztikus függvénnyel.
3. Vizualizálja a feltárt állapotgráfot 3D-ben, erőalapú elrendezést (force-directed layout) és Barnes–Hut optimalizációt [2] használva.
4. Lehetővé teszi a kézi játékot és az állapotok interaktív vizsgálatát a gráfcsúcsokra való kattintással.

3.1.2. Funkcionális leírás

A funkcionális követelményeket a 3.1. táblázatban található felhasználói történetek (user stories) foglalják össze.

Felhasználói történetek			
Cél	**Előfeltétel**	**Esemény**	**Eredmény**
Kiválasztani egy pályát	A feladvány választó fül meg van nyitva	Rányomok a megfelelő pálya előnézet rublikájára	Betöltődik a pálya kezőállapota, a állapotgráfon csak a kezdőállapot látszik, átvált a jobb panel az algoritmus beállítások fülre
Felfedezni az állapotgráf csúcsait	Kereső algoritmus beállítások fül meg van nyitva	Rányomok a csúcsfelfedezés gombjára	Az állapotgráfon felvesszük a felfedezett csúcsokat, a célcsúcsok ki vannak emelve
Megállítani a csúcsfelfedezést	Épp fut csúcsfelfedezés	Rányomok a csúcsfelfedezés megállítása gombjára	A csúcsfelfedezés megáll
Kiválasztani milyen keresőalgoritmust legyen kiválasztva	Kereső algoritmus beállítások fül meg van nyitva	Kiválasztom a megfelelő keresőalgoritmust az opciók közül	A keresőalgoritmus beállításra kerül
Kiválasztani milyen heurisztikus függvény legyen kiválasztva	Kereső algoritmus beállítások fül meg van nyitva	Kiválasztom a megfelelő heurisztikus függvényt az opciók közül	A heurisztika beállításra kerül
Futtatni a beállított keresőalgoritmust	Kereső algoritmus beállítások fül meg van nyitva	Rányomok a keresőalgoritmus alkalmazása gombra	A kiválasztott beállítások alapján alkalmazva lesz a kereső algoritmus a betöltött feladványra

Cél	Előfeltétel	Esemény	Eredmény
Elindítani/ folytatni a keresőalgoritmust	A játéktábla fül meg van nyitva	Rányomok a keresőalgoritmus indítása/folytatása gombjára	A keresőalgoritmus elindul, a gráfon ki vannak emelve a globális munkaterületen tárolt csúcsok és élek, ha a keresőalgoritmus talál célállapotot, akkor leáll az algoritmus
Megállítani a keresőalgoritmust	A játéktábla fül meg van nyitva és épp fut keresőalgoritmus	Rányomok a keresőalgoritmus megállítása gombjára	A keresőalgoritmus megáll, a gráfon a globális munkaterületen tárolt csúcsok és élek ki vannak emelve
Lépésenként futtatni a keresőalgoritmust	A játéktábla fül meg van nyitva	Rányomok a keresőalgoritmus lépésenkénti futtatása gombjára	A keresőalgoritmus egy lépést futtat, a gráfon ki vannak emelve a globális munkaterületen tárolt csúcsok és élek
Beállítani a keresőalgoritmus sebességét	A játéktábla fül meg van nyitva	A sebesség állító csúszkát a megfelelő helyre húzom	A keresőalgoritmus minden lépése a beállított sebességgel történik
Manuálisan lépni a táblán	A játéktábla fül meg van nyitva	A játéktáblán kattintok a lépésnek megfelelő piros nyílra	A játéktáblán elmozdul az autó, az állapotgráfon felfedezzük, kitüntetjük az új állapot csúcsát
A játéktáblát manuális módba váltani	A játéktábla fül meg van nyitva	A játéktábla fölötti manuális gombra kattintok	A játéktábla manuális módba vált, az állapotgráfon ki van tüntetve a megjelenített állapot, ha futott algoritmus az a háttérben fut tovább

Cél	Előfeltétel	Esemény	Eredmény
Az algoritmus lépéseit megjeleníteni a táblán	A játéktábla fül meg van nyitva	A játéktábla fölötti algoritmus gombra kattintok	A játéktábla algoritmus módba vált, a táblán az algoritmus által legutóbb kiterjesztett állapot van megjelenítve, a játéktábla mutatja az algoritmus újjab kiterjesztett állapotait
Egy felfedezett állapotba ugrani	Egy feladvány be van töltve	Rányomok (M1) egy felfedezett állapot csúcsára az állapotgráfon	A játéktábla a kiválasztott állapotot mutatja, a gráfon a kiválasztott csúcs ki van emelve
Forgarni az állapotgráfot	Egy feladvány be van töltve	Az állapotgráfon kattintok (M2) és húzom az egeret	Az állapotgráf forog az aktív csúcs körül az egér mozgásának megfelelően
Mozgatni az állapotgráfot	Egy feladvány be van töltve	A WASD gombok közül nyomom valamelyiket	Az állapotgráf elmozdul a vásznon a gombnak megfelelően
Az állapotgráfon elrejteni/megjeleníteni a nem releváns éleket, csúcsokat	A játéktábla fül meg van nyitva	Rákattintok az irreleváns élek és csúcsok elrejtésének kapcsolójára	Az állapotgráfon elrejtődnek/megjelennek az irreleváns élek, csúcsok
Követni az állapotgráfon az algoritmus által kiterjesztett csúcsokat	A játéktábla fül meg van nyitva	Rákattintok az algoritmus csúcsai követésének kapcsolójára	Az állapotgráf kamerája követi az algoritmus által kiterjesztett csúcsokat

Cél	Előfeltétel	Esemény	Eredmény
Nagyítani/ kicsinyíteni az állapotgráfot	Egy feladvány be van töltve	Az állapotgráfon az egér görgőjét használom	Az állapotgráf nagyítódik/kicsinyítódik az egér görgőjének megfelelően
Kilépni a programból	A program fut	Bezárom a program ablakát	A program bezárul

3.1. táblázat. Felhasználói történetek

3.1.3. Nem-funkcionális követelmények

- **Teljesítmény**: a 3D gráfnak interaktívnak kell maradnia akár több ezer csúcs esetén is. A Barnes–Hut optimalizációnak kezelhető szinten kell tartania a képkockánkénti számítási igényt.
- **Hordozhatóság**: önálló futtatható fájlok Windowsra és Linuxra; fordíthatóság más platformokra a Godot szerkesztőn keresztül.
- **Használhatóság**: intuitív felhasználói felület színkódolt gráfelemekkel és reagáló vezérlőkkel.
- **Helyesség**: a keresőalgoritmusoknak az elvárt tulajdonságok szerint kell működniük (pl. az A^C gráfkeresésnek monoton heurisztikával optimális megoldást kell találnia).

3.2. Tervezés

3.2.1. Szoftverarchitektúra

Az alkalmazás a **Modell–Nézet (Model–View)** architektúrát követi:

Modell réteg (`src/Model/`) Tiszta C# osztályokat tartalmaz a játékállapot reprezentációjára, a lépésgenerálásra, a heurisztikus kiértékelésre és a keresőalgoritmusokra. A modell nem függ a Godot motortól, egyedül a Godot-ban definiált `Vector2I` struktúrát használja, hogy ne kelljen később konvertálni azt.

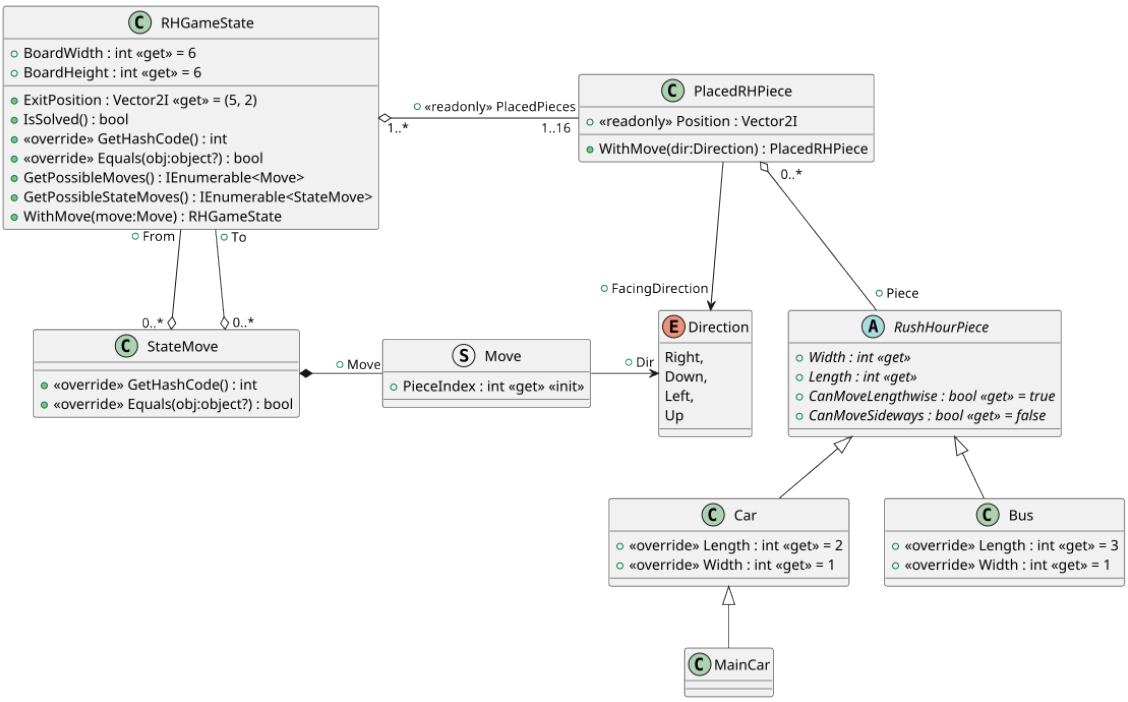
Nézet réteg (`src/Nodes/`) Olyan Godot node scripteket tartalmaz, amelyek a motor alaposztályaiból származnak (`Control`, `RigidBody3D`, `MeshInstance3D`,

stb.). Ezek az osztályok kezelik a kirajzolást, a fizikai szimulációt, a felhasználói felületet és bevitelt. Feliratkoznak a modell eseményeire (`NewEdge`, `PathChange`, `NewCurrent`, `Terminated`), hogy reagáljanak az algoritmus előrehaladására.

A modell és a nézet közötti kommunikáció az absztrakt `RHSolver` alaposztályon definiált C# eseményeken (events) keresztül valósul meg. A nézet feliratkozik ezekre az eseményekre, és létrehozza vagy frissíti a megfelelő vizuális elemeket (csúcsok, élek, kiemelések).

3.2.2. Modell

A modell fontosabb részeinek az osztálydiagramja a 3.1. és 3.2. ábrán látható.



3.1. ábra. Az állapot ábrázolás osztálydiagramja

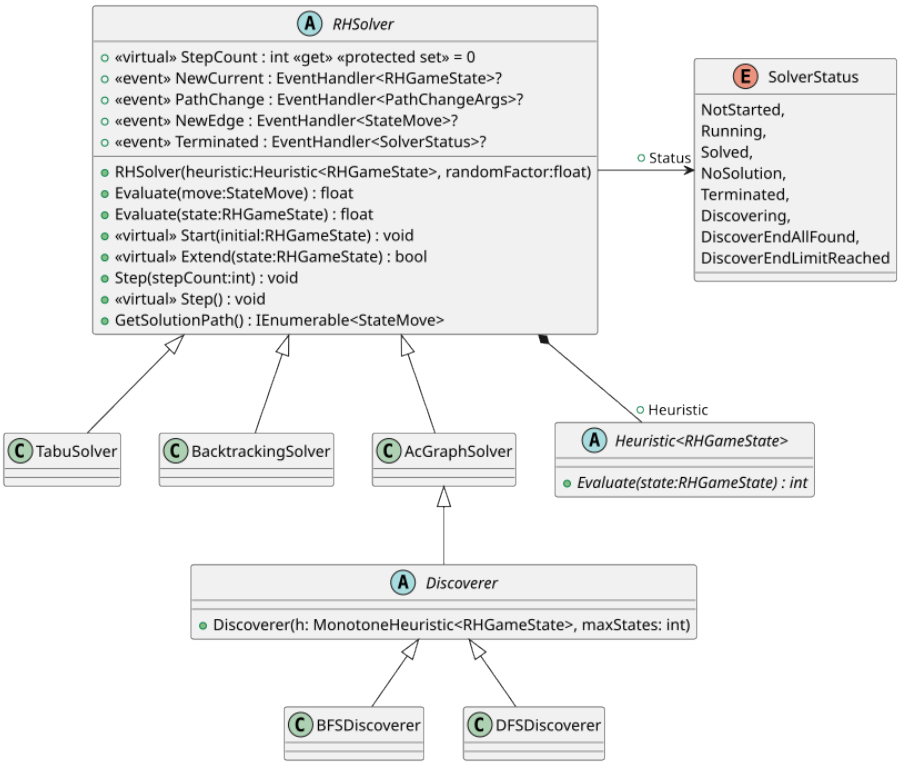
Az állapotok és bábuk mind módosíthatatlan osztályok.

`RHGameState` Egy táblakonfiguráció módosíthatatlan reprezentációja, a módosíthatatlanságot az `init;` kulcsszóval és a *readonly* adat struktúrákkal biztosítjuk. Az elhelyezett járműveket `PlacedRHPiece` objektumok tömbjét tárolja, ezekből pedig generál egy 6×6-os egész értékű mátrixot tárol a gyors mező

eléréshez. Biztosítja a `GetPossibleMoves()`, `WithMove()`, `IsSolved()` metódusokat, valamint érték-alapú egyenlőségvizsgálatot amit hash kódokkal van felgyorsítva. Mivel nem módosítható az adatszerkezet, ezért elég csak inicializáláskor egyszer kiszámolni a hash kódját az állásra, és később mindig ezt adja vissza.

`PlacedRHPiece` Összekapcsol egy `RushHourPiece`-t (Car, MainCar vagy Bus) egy táblapozícióval és iránnyal. A `WithMove()` metódus egy új példányt ad vissza frissített pozícióval.

`Move` / `StateMove` A `Move` egy önmagában nem sok információt hordozó struktúra (járműindex + irány). A `StateMove` egy teljes átmenetet rögzít: forrásállapot, célállapot és az előbb említett lépést. Mivel a modellünkben minden lépésnek meg lehet tenni a fordítottját is, ezért egyenlőségvizsgálatnál nem teszünk különbséget a fordított lépéssel: $(A \to B) = (B \to A)$, így irányítatlan gráfot alkotunk a modellünkhöz.



3.2. ábra. A keresőalgoritmusok osztálydiagramja

`RHSolver` **hierarchia** Az absztrakt `RHSolver` alaposztály definiálja az eseményvezérelt interfészt és a közös logikát. Konkrét alosztályok:

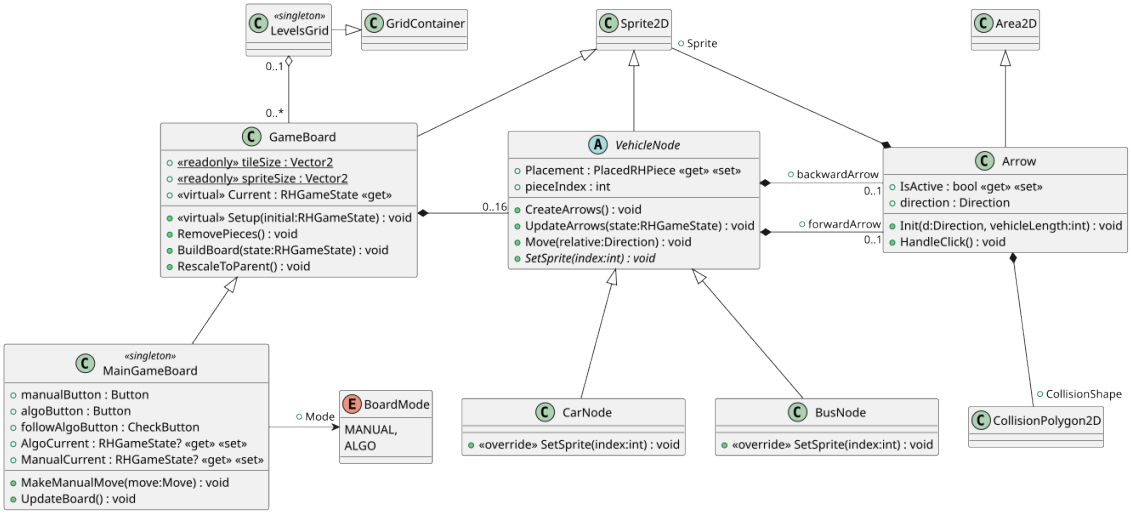
- `TabuSolver`: fenntart egy listát a legutóbbi állapotokról; kiválasztja a legjobb nem-tabu szomszédot.
- `BacktrackingSolver`: rendezett opciólisták vermét tartja fenn; visszalép zsákutcák esetén.
- `AcGraphSolver`: $f = g + h$ szerint rendezett `PriorityQueue`-t használ; egy szótárban követi a felfedezett állapotokat.
- `BFSDiscoverer` / `DFSDiscoverer`: a `Discoverer` (a gráfkeresést veszi alapul ezért a `AcGraphSolver`-ből öröklődik) osztály nem ellenőrzi és terminál megoldott állapot kiterjesztése esetén, a szélességi és mélységi felfedező alosztályai a kiértékelő függvényt definiálják felül.

`Heuristic` **hierarchia** Absztrakt osztályhierarchia, amely a típusrendszeren keresztül kényszeríti ki a heurisztika tulajdonságait: `Heuristic` $\supset$ `NonNegativeHeuristic` $\supset$ `AdmissibleHeuristic` $\supset$ `MonotoneHeuristic`. Az A^C gráfkereső fordítási időben megköveteli a `MonotoneHeuristic` típust.

Még érdemes megemlíteni a modellhez tartozó `Levels` Statikus osztályt, amely a szinteket leíró sztringeket és egy feldolgozó metódust tartalmaz, amely `RHGameState` példányokká alakítja azokat. Ezzel könnyen leírható egy pálya anélkül, hogy az objektumok pozíció struktúráját kéne megadni. A szintek hivatalos videójáték feladványaiból lettek válogatva. [6]

3.2.3. Nézet

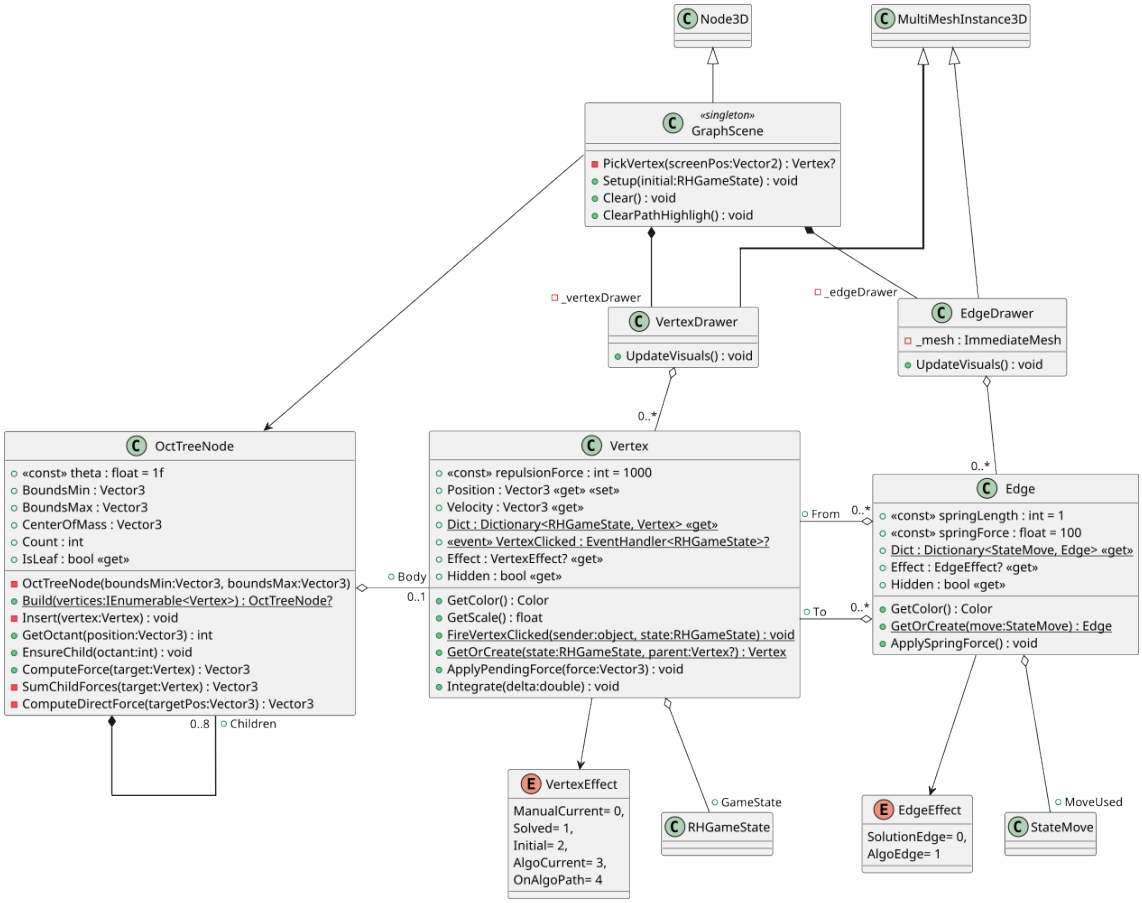
A nézet kiemelt részének osztálydiagramja a 3.3. és 3.4. ábrákon látható.



3.3. ábra. Az játéktábla megjelenítéséhez használt osztályok osztálydiagramja

`GameBoard` Egy `RHGameState` megjelítéséért felelős objektum a szintbetöltés fülön a feladványok előnézetei ezt használják. Az játéktábla fülön egy `MainGameBoard` alosztály van példányosítva, ami ezeli a Kézi/Algoritmus mód váltását, a táblaállapot megjelenítését és a járművek nyilaiért felel. Feliratkozik a csúcskattintási eseményekre a manuális navigációhoz.

`VehicleNode` / `CarNode` / `BusNode` 2D sprite-ok, amik a járműveket reprezentálják, egy GameBoard objektumhoz tartoznak (A Godot scene tree gyerek/szülő elérő metódusokon keresztül tudunk hivatkozni a kompozíció tagjaira). Ha a `MainGameBoard` hoz tartozik, akkor `Arrow` léptető nyíl objektumok tartoznak hozzá a kézi mozgatáshoz. A nyilak csak egyszer vannak példányosítva, és ha nem lehet a nyíl által mezőre lépni, akkor az `IsActive` mező alapján ki lesz kapcsolva a láthatósága és interakciója.



3.4. ábra. Az állapotgráf megjelenítéséhez használt osztályok osztálydiagramja, a nyolcasfa megvalósítása a projekthez szabva

`GraphScene` A 3D gyökér-node, amely futtatja a szimulációt, meghívja a kirajzolást, és kezeli a nehéz számítással járó lassulást. Minden fizikai képkockában újjáépí-

ti a Barnes–Hut nyolcasfát a csúcsok aktuális helyzete szerint. Minden fizikai képkockánál minden él és csúcs erőkiszámítását futtatja párhutamosan, majd a csúcsok elmozdulását is párhuzamosítva kiszámítja. A csúcsok kattintásához lefuttat egy ray castot a kamera aktuális állása szerint.

`Vertex` Egy állapotot jelöl, egy körként van megjelnítve az állapotgráfon. Szálbiztos adatstruktúrát biztosít a rá gyakorolt erők akkumlálására. Prioritás-alapú effekt-rendszert tart fenn a színhez és mérethez. A felépített nyolcasfán kereszül approximálja a rá ható erőt.

`Edge` Egy lépést jelöl, egy vonalként van megjelenítve az állapotgráfon. Az `ApplySpringForce` metódussal lehet a csúcsaira hozzáadni a rugóerőt. Lekérdezhető a színe a beállított effektusa alapján.

`VertexDrawer, EdgeDrawer` A renderelésért felelős osztályok, ahelyett, hogy minden csúcs él, külön lenne `Sprite3D` vagy `ImmediateMesh` objektum lenne, összegyűjtjük a komponenseket egy-egy tömbbe és azt felhasználva egyszerre rajzoljuk ki az összes csúcsot, élt, ezzel jelentősen felgyorsítva a renderelést.

`OctTreeNode` A *NetOctree* repository [7] alapján lett implementálva, csak a projekthez szükséges metódusokkal, és kiegészítve erőapproximálással. Implementálja a Barnes–Hut algoritmust három dimenziós térre.

Még említésre való osztályok a nézet rétegben:

`Camera3d` A `GraphScene`-hez tartozik ortogonális projekciójú 3D kamera mozgással, forgatással és nagyítással az állapotgráf részletesebb megtekintéséhez. A *3D-RTS-Camera* repository alapján készült. [8]

`AlgoPlayer` Vezérli az algoritmus futásának időzítését, feliratkozik/leiratkozik a solver eseményeiről, kezeli a lejátszást/szünetet/léptetést/újraindítást.

`SolverSettingsTab` Felhasználói felület az algoritmus- és heurisztika-választáshoz, tabu mérethez, véletlenszerűséghez és a BFS/DFS felfedezéshez.

3.2.4. Adattárolás

Az alkalmazás nem használ adatbázist. A rejtvények szintjei beégetett sztring literálokként tárolódnak a `Levels` osztályban. Minden szint egy 7 soros sztring: az

első sor a szint címe, a maradék hat sor pedig a 6×6-os rácsot reprezentálja. Ez a reprezentáció kompakt, ember által olvasható és egyszerűen interpretálható.

3.2.5. Keresőalgoritmusok és heurisztikák

Ez a szakasz részletezi a három keresőalgoritmust és az irányításukhoz használt heurisztikus függvényeket.

Heurisztikus függvények

Mind a négy heurisztika megfelel három alapvető tulajdonságnak:

- **Nemnegativitás**: $h(s) \geq 0$ minden s állapotra.
- **Megengedhetőség (Admissibility)**: $h(s) \leq h^*(s)$, ahol $h^*(s)$ a megoldáshoz szükséges minimális lépésszám s-ből.
- **Monotonitás** (konzisztencia): bármely s állapotra és annak egyetlen lépéssel elérhető s' utódjára $|h(s) - h(s')| \leq 1$.

Ezek a tulajdonságok szükségesek, hogy a gráfkeresés egyben A^C gráfkeresés legyen, mivel ekkor egy állapot első kiterjesztésekor a hozzá vezető legrövidebb utat már megtaláltuk, többet nincs szükség kiterjeszteni azt az állapotot.

Null heurisztika. $h(s) = 0$ minden állapotra. Nem ad információt az állapotról; a keresés nem informált brute-force felfedezéssé válik. Hasznos az alap algoritmus-viselkedés megfigyeléséhez.

Az alábbi heurisztikák mind speciálisan a Rush Hour 6×6-os táblájára és a 3. sor jobb szélén lévő kijáratára lettek implementálva.

Távolság (Distance) heurisztika. $h(s) = w_{\text{tábla}} - 1 - x_{\text{fő}}$, ahol $w_{\text{tábla}}$ a tábla szélessége $x_{\text{fő}}$ a főautó elejének oszlopa (0-tól indexelve). 6 mező széles táblára ez az érték 0 (megoldva) és 4 között mozog. Ez megengedhető, mivel a főautónak legalább ennyi mezőt kell elmozdulnia a kijáratig. Monoton, mert egyetlen lépés legfeljebb 1-gyel változtatja meg a főautó távolságát.

Távolság + torlaszok heurisztika. A távolság heurisztikához hozzáad egy-egy költséget minden olyan jármű által elfoglalt mezőért, amely a főautó és a kijárat között van a kijárat sorában. Megengedhető, mert minden blokkoló elem eltávolí-

tása legalább egy plusz lépést igényel. Monoton, mert egy lépés legfeljebb 1-gyel módosítja a távolságot vagy a torlaszok számát.

Legkevesebb lépés (Least-moves) heurisztika. Kiterjeszti a távolság heurisztikát a blokkoló járművek eltávolítási költségének alsó becslésével. Minden, a főautó útját elálló járműre kiszámítja a minimális lépésszámot, amellyel félretolható, figyelembe véve: (a) az irányt, amerre tolni kell (buszoknál mindig lefelé a 4–6. sorokba, mivel a kijárat a 3. sorban van), és (b) a tolást akadályozó további járműveket. Ez az elemzés a monotonitás megőrzése érdekében már nem lehet ugyanígy kiterjeszteni a fő autót blokkoló autóknak a blokkolóinak eltolánsának lépésszámára, mivel ugyanezzel a becslési módszerrel már nem lenne monoton a heurisztika, egy ilyen autó elmozdítása 2-t is csökkenthet a heurisztikus függvény értékén.

Tabu keresés

A tabu keresés egy lokális keresési meta-heurisztika. Minden lépésben felfedezi az összes szomszédos állapotot, kiszűri a tabu listán szereplőket, kiértékeli mindegyikre a heurisztikus függvényt és a heurisztika szerinti legjobb maradék szomszédra lép.

A tabu lista az utolsó n meglátogatott állapotot tárolja. Főbb konfigurációk:

- $n = 0$: mohó keresés memória nélkül, mindig a heurisztika szerinti legjobb szomszédra lép.
- $n = 1$: hegymászó algoritmus; csak a közvetlenül megelőző állapotba való visszatérést kerüli el.
- $n > 1$: elkerüli az utolsó n állapotot. Megoldás nélkül leáll, ha minden szomszéd benne van a tabuban.

A tabu keresés *nem* garantálja a megoldás megtalálását, sem annak optimalitását. Ez egy gyors, lokális módszer, amely csapdába eshet az állapottér olyan régióiban, ahol kedvezőek a heurisztikus pontszámok, de nincs megoldás.

Visszalépéses (Backtracking) keresés

A visszalépéses keresés minden állapothoz a szomszédos még az úton még azelőtt nem látogatott állapotok heurisztika szerint rendezett sorozatát tartja fenn: minden lépésben a jelenlegi út utolsó állapotának a legjobb még nem próbált opciót vizsgálja meg. Ha már nincs új opció akkor leveszi az állapotot feléített út végéről, azaz algoritmus visszalép.

Leállási garancia: a backtracking algoritmus véges állapotú állapotgráfra garantáltan leáll. Ha megoldás nélkül áll le, akkor a kezdőállapotból nem érhető el megoldás. Ugyanakkor csak az aktuális útvonalon lévő állapotokat kerüli el, a korábban már bejártakat nem. Ez azt jelenti, hogy többször is feltárhat nagy állapotcsoportokat, ami jelentős redundáns munkához vezethet.

Időbonyolultság: legrosszabb esetben a backtracking $O(V!)$ útvonalat jár be, ahol V az elérhető állapotok száma. Pontosabban, minden egyszerű utat feltárhat a kezdőállapotból. Egy b elágazási tényezőjű és V állapotú térben a legrosszabb eset $O(b^V)$. A Rush Hour esetében egy adott feladványra az elágazási tényező legfeljebb $2N_{\text{jármű}}$ ahol $N_{\text{jármű}}$ az autók és buszok száma összesen. Megfelelő heurisztikával az algoritmus a legtöbb esetben ennél sokkal gyorsabban talál megoldást, amennyiben elérhető.

$\mathbf{A^C}$ gráfkeresés

Az A^C gráfkeresés egy informált keresőalgoritmus, amely az $f(n) = g(n) + h(n)$ kiértékelő függvényt használja, ahol $g(n)$ a mélység, $h(n)$ pedig a heurisztikus pontszám.

Mivel minden alkalmazható heurisztika nemnegatív, megengedhető és monoton, a következő tulajdonság teljesül: amikor egy állapotot kiveszünk a prioritási sorból kiterjeszteni, az az útvonal, amelyen elértük, a legrövidebb út az adott állapothoz. Ezért minden állapotot legfeljebb egyszer kell kiterjeszteni (az A^C-ben a „C” a „closed list”-re utal – a már kiterjesztett állapotokat nem terjesztjük ki újra).

Ez kiküszöböli a backtracking körbejárási és újrafelfedezési problémáit, de az algoritmus sok olyan állapotot is feltárhat, amelyek nem szükségesek a megoldáshoz, mivel megvizsgál minden állapotot, amelynek f-értéke kisebb az optimális megoldás költségénél.

Időbonyolultság: Az A^C gráfkeresés minden állapotot legfeljebb egyszer terjeszt ki, ami legfeljebb $O(V)$ kiterjesztést jelent. Minden kiterjesztés legfeljebb $O(b)$ utódot generál (b az elágazási tényező, max $2 \times N_{\text{jármű}}$), és minden beillesztés a prioritási sorba $O(\log V)$ költségű. Összesen: $O(V \cdot b \cdot \log V)$. A tárigény $O(V)$ a felfedezett és kiterjesztett állapotok szótára miatt.

Barnes–Hut algoritmus és a nyolcasfa

Az erőalapú gráfelrendezés megköveteli a taszítóerők kiszámítását minden csúcspár között, ami képkockánként $O(V^2)$. Több ezer csúcsnál ez teljesítménybeli szűk keresztmetszetté válik.

A Barnes–Hut algoritmus [2] ezt $O(V \log V)$-re csökkenti egy térbeli hierarchikus adatszerkezet – 3D-ben egy **nyolcasfa (octree)** – használatával. Az algoritmus menete:

1. **Nyolcasfa felépítése**: kiszámítjuk az összes csúcsot befoglaló tengelypárhuzamos kockát, azaz amibe minden csúcs belefér, ezt vesszük alapul és sorra beillesztjük a csúcsokat. Minden belső csomópont tárolja a részfájában lévő csúcsok számát és tömegközéppontját. A levelek egyetlen csúcsot tartalmaznak. A felépítés V csúcsra $O(V \log V)$ időt igényel.[2]

2. **Erők kiszámítása**: minden v csúcsra meghívjuk a fa gyökerére az erőkiszámítást. Minden o csomópontnál:

   - Ha $s/d < \theta$ (ahol s az o kockájának oldalhosszúsága, d pedig v távolsága o tömegközéppontjától), akkor az egész cellát egyetlen testként kezeljük, amelynek tömege megegyezik a benne lévő csúcsok számával. Ez a *Barnes–Hut kritérium.*
   - Ellenkező esetben rekurzívan bejárjuk o csomópont gyerekeit, minden gyerekre kiszámítva/approximálva a v csúcsra ható erőt és ezeket applikáljuk a v csúcsra

   Az eljárás egy csúcsra átlagosan $O(\log V)$ időt igényel. [2] Minden csúcsra összesen az erőkiszámítás is $O(V \log V)$ időt igényel.

3. A tipikusan $0.5 - 1.5$ intervallumra beállított θ paraméter szabályozza a pontosság és a sebesség közötti kompromisszumot. Az alkalmazás a $\theta = 1.0$ értéket használja.

A két pont közötti taszítóerő a fordított négyzetes törvényt követi: $\vec{F} = -k \cdot m/d^2 \cdot \hat{r}$.

A nyolcasfát minden fizikai frissítésnél elölről felépítjük, mivel a csúcsok pozíciója folyamatosan változik. A faépítés $O(V \log V)$, az erő számítása csúcsonként $O(\log V)$, így a teljes időkomplexitás $O(V \log V)$ frissítésként.

3.3. Implementáció

3.3.1. Felhasznált technológiák

- **Godot Engine 4.6** (Mono/.NET variáns) [3]: nyílt forráskódú játékmotor a 2D/3D rendereléshez, fizikához és felhasználói felülethez.
- **C# és .NET SDK 10.0** [4]: az összes alkalmazáslogika ezen a nyelven készült. Az egyike a Godot által támogatott nyelvek közül és ideális az objektum orientált logikához.

Kihasznált godot funkciók:

- `MultiMeshInstance3D` A csúcsok és élek batchelt rendereléséhez.
- `Container` objektumok alkalmazása a felhasználói felület felépítéséhez. Beépítetett gombra, `CollisionShape2D`-re kattintási események használata.
- `_PhysicsProcess` metódus futtatása háttérszálon, hogy az interakciót ne blokkolja.

3.3.2. Eltérések az eredeti tervtől

Erő kiértékelés és alvási optimalizáció. A fejlesztés során először minden csúcs és él egy saját megjelenítéssel rendelkező godot objektum volt. A több ezer él és csúcs külön meghívott megjelenítése minden képkockán rendkívül számításigényesnek bizonyult. Ennek megoldására először az alábbi megoldás került implementációra:

1. Ahelyett, hogy az erőket közvetlenül a Godot fizikai API-ján keresztül alkalmaznánk, az erők egy csúcsonkénti (`Vector3 _pendingForces`) mezőben gyűlnek.
2. Minden fizikai frissítés végén egy `Processor` node végigmegy a csúcsokon meghívja a kiértékelő függvényüket.
3. Egy dinamikus küszöbérték dönti el, hogy a függőben lévő erő elég jelentős-e az alkalmazáshoz. Ez a küszöb a csúcs „életkorával” (`_timeAwake`) és az összes csúcs számával skálázódik, így nagy, stabilizálódott gráfoknál a kis maradék erőket figyelmen kívül hagyja, a jelentős erőket pedig meghívja a Godot függvényeivel.
4. Ha az erők tartósan a küszöb alatt maradnak, a csúcsokra beállított sebesség tompítás miatt el fogják érni a nyugalmi állapotot, ekkor a motor „alvó” módba teszi a `RigidBody3D`-t, leállítva a mozgását és a `_PhysicsProcess` hívását,

ezután a csúcs újra felébredhet, ha egy élnek a rugóereje a küszöb fölé kerül és applikálva lesz az erő a csúcsra.

5. Azok az élek, amelyeknek mindkét végpontja alszik, kihagyják a vonal-mesh frissítését, jelentős renderelési terhet levéve a rendszerről.

Ez a lépcsőzetes optimalizáció (erőküszöb → fizikai alvás → render-kihagyás) jelentős teljesítménynövekedést eredményezzet, amikor az állapotgráf már a nyugalmi helyzetébe rendeződött. Viszont a legtöbb többezer csúcsos állapotgráfra a nyugalmi helyzet elérése túl hosszú volt ahhoz, hogy aktuális lehessen ez az optimalizáció, ezért ezt teljesen újra kellett gondolni és átalakítani a csúcsokat és éleket nem godotból származó osztályokká, és a kirajzolásukat kiszervezni külön osztályokba. Ez az optimalizáció már aktívan mozgó állapotgráfra is sokkal gyorsabb renderelést eredményezett. Ezen felül még több ezer csúcs esetén, a fizikai frissítések számát levisszük $60/s \rightarrow 10/s$-ra, így nem kell az alkalmazásnak bevárnia minden UI frissítéshez az elvárt egy fizikai frissítés lefutását, hanem több UI képkocka lefuthat fizikai frissítések között, és így reszponzív tud maradni a felhasználói felület.

3.4. Tesztelés

3.4.1. Egységtesztek

A következő egységtesztek ellenőrzik a program helyességét:

Egységtesztek		
#	**Teszteset**	**Ellenőrzés**
U1	Szint-parzolás	Minden szint sikeresen betöltődik, a járművek száma és pozíciója egyezik az elvárttal.
U2	Táblarács konzisztencia	Parzolás után a 6×6-os rács helyesen rendeli a cellákat a járművekhez vagy -1-hez.
U3	Lépésgenerálás	A `GetPossibleMoves()` csak érvényes lépéseket ad vissza (nincs átfedés, nincs kilépés a tábláról).
U4	Állapot immutabilitás	A `WithMove()` új állapotot ad vissza; az eredeti állapot változatlan marad.

#	**Teszteset**	**Ellenőrzés**
U5	Állapot egyenlőség	Két azonos járműpozíciójú állapot egyenlő; az eltérőek nem.
U6	Hash konzisztencia	Az egyenlő állapotok hash kódja azonos; a különbözőeké (nagy valószínűséggel) eltérő.
U7	Megoldás detektálása	Egy állapot akkor és csak akkor megoldott, ha a főautó pozíciója $(5, 2)$.
U8	Null heurisztika	Minden állapotra 0-t ad vissza.
U9	Távolság heurisztika tartomány	Az érték $[0, 4]$ között van; csak megoldott állapotra 0.
U10	Heurisztika megengedhetőség	Minden heurisztikára: $h(s) \leq$ tényleges minimális lépésszám (ismert feladványokon mérve).
U11	Heurisztika monotonitás	Minden s állapotra és s' utódra: $h(s) \leq h(s')+1$.
U12	Tabu solver leállása	Megfelelő tabu mérettel egy megoldható feladványon a solver vagy megoldást talál, vagy jelzi a leállást.
U13	Backtracking teljesség	Megoldható feladványon a backtracking megtalálja a megoldást. Ha a feladványra megoldhatatlan jelzi azt.
U14	A^C optimalitás	Adott feladványra ismert optimális hosszúságú megoldással az A^C monoton heurisztikával optimális hosszúságó megoldást talál.
U15	StateMove szimmetria	$(A \rightarrow B)$ megegyezik $(B \rightarrow A)$-val az élek irányítatlansága miatt.
U16	BFS/DFS felfedezési szám	Állapotlimit esetén a felfedezés pontosan a megadott számnál áll le.

3.2. táblázat. Egységtesztek a programhoz

Az egységtesztek futtatásához a projekt könyvtárában az alábbi parancsot kell futtatni:

```
dotnet add package gdUnit4.api --version 5.1.0-rc1
```

Ezután a szerkesztőből futtathatóak az egységtesztek.

3.5. ábra. Egységtesztek sikeres futtatása a szerkesztőben

3.4.2. Végfelhasználói tesztesetek

A következő tesztesetek a 3.1. táblázat felhasználói történetein alapulnak.

Végfelhasználói tesztesetek				
#	**Művelet**	**Előfeltétel**	**Bemenet**	**Elvárt eredmény**
E1	Szint betöltése	Nem fut algoritmus	„Level 1” gomb	Tábla mutatja az 1. szintet; a gráfon egy kék csúcs van.
E2	Kézi lépés	Szint betöltve	Kattintás egy autó előre nyilára	Az autó elmozdul; új csúcs és él jelenik meg a gráfon.

#	**Művelet**	**Előfeltétel**	**Bemenet**	**Elvárt eredmény**
E3	BFS felfedezés	Szint betöltve	Max áll.: 50; BFS indítás	A gráf feltöltődik max. 50 csúccsal; a megoldások zöldek.
E4	Felfedezés leállítása	Felfedezés fut	Megállítás (*Pause*) gomb	A folyamat megáll; a csúcsok megmaradnak.
E5	Tabu keresés futtatása	Szint betöltve	Tabu: 5, Távolság heur.; Play	Algoritmus fut, narancs/sárga kiemelések; státusz: „Running”.
E6	Algoritmus léptetése	Algoritmus nem terminált	Léptetés (*Step*) gomb	Egy lépés lefut; a lépésszámláló nő.
E7	Megoldás megtalálása	Futás megoldható szinten	Kivárás	Státusz: „Solved”; megoldási út zöld; megoldási hossz látható.
E8	Ugrás csúcsra	Egy szint be van töltve, valamennyi csúcs fel van fedezve	Bal klikk egy csúcsra	Tábla Kézi módba vált az adott állapottal; a csúcs piros lesz.
E9	Kamera forgatása	Gráf látható	Jobb egérgomb + húzás	A kamera forog a gráf körül.
E10	Nagyítás	Gráf látható	Egérgörgő	A nézet közelít vagy távolít.
E11	Kamera mozgatása	Gráf látható	W/A/S/D billentyűk	A kamera elmozdul az lenyomott gomb szerint.

#	**Művelet**	**Előfeltétel**	**Bemenet**	**Elvárt eredmény**
E12	Csúcsok elrejtése	Gráf látható	Hide kapcsoló	Minden fehér csúcs és él el lesz rejtve.
E13	Solver újraindítása	Futott már az algoritmus	Restart gomb	Algoritmus alaphelyzetbe áll; algoritmus kiemelések és lépésszám törlődnek.
E14	Módváltás (Kézi/Algo)	Szint betöltve	Manuális/Algoritmus gomb	A tábla a manuális vagy az algoritmus módba vált.

3.3. táblázat. Felhasználói történeteken alapuló tesztesetek

4. fejezet

Összegzés

A szakdolgozat a Rush Hour logikai játék keretein belül valósította meg a keresőalgoritmusokat, heurisztikákat, a megjelenítést és az interakciót. Ezen kívül számos különböző problémára, játékra lehet érdemes hasonló elemzéseket elvégezni. Az állapottérbeli keresésre 3 alapvető algoritmus került implementálásra, amikkel jól ábrázolhatóak a különböző megközelítési egy problémának. Arra is alkalmas a rendszer, hogy egy saját algoritmust látványosan tudjunk futtatni, és elemezzük annak viselkedését, ellenőrizzük az elvárt helyességét.

Bővítési lehetőségek. Számos területen lehet még tovább fejlesztheti az alkalmazást:

- A Rush Hour-ön túlmutató további rejtvénytípusok támogatása a keretrendszer általánosítása érdekében.
- Az A^* keresés implementálása (a lezárt állapotok újranyitásával) az A^C algoritmussal való összehasonlítás céljából.
- Párhuzamos algoritmusfuttatás a különböző konfigurációk egymás melletti összehasonlításához.
- GPU-gyorsított erőszámítás a több ezer csúcsból álló gráfok kezeléséhez.
- Algoritmus által megtalált megoldás lejátszása a játék táblán.
- Visszalépéses keresés mélységi korlátának implementálása.
- Az alkalmazás átstruktúrálása MVVM (Model-View-ViewModel) architektúrába.

Irodalomjegyzék

[1] ThinkFun Inc. *Rush Hour — Traffic Jam Logic Game*. https://www.thinkfun.com/products/rush-hour/. Accessed: 2026-04-21. 1996.

[2] Josh Barnes és Piet Hut. „A hierarchical O(N log N) force-calculation algorithm". *Nature* 324.6096 (1986), 446–449. old. DOI: 10.1038/324446a0.

[3] Godot Engine contributors. *Godot Engine – Free and open source 2D and 3D game engine*. https://godotengine.org/. Version 4.6, Accessed: 2026-04-21. 2026.

[4] Microsoft. *.NET 10 SDK*. https://dotnet.microsoft.com/. Accessed: 2026-04-21. 2025.

[5] Tom Ventimiglia és Kevin Wayne. *Barnes-Hut Galaxy Simulator*. 2003. URL: https://www.cs.princeton.edu/courses/archive/fall03/cs126/assignments/barnes-hut.html.

[6] ThinkFun. *Rush Hour® Deluxe – The ultimate traffic jam game!* https://store.steampowered.com/app/2073760/Rush_Hour_Deluxe__The_ultimate_traffic_jam_game/. 2022.

[7] Cserép Máté. *A dynamic octree implementation written in C# as a .NET Standard 2.1 library, built on the System.Numerics library*. https://github.com/mcserep/NetOctree/.

[8] Emil Hakala. *A fully customizable and easy-to-use 3D RTS-style Camera for Godot 4.3*. https://github.com/EmilHakala/3D-RTS-Camera.

Ábrák jegyzéke

2.1. A projekt megnyitása és futtatása a Godot szerkesztőből 7
2.2. A projekt exportálásának lépései a Godot szerkesztőből 8
2.3. Az alkalmazás egy algoritmus futása közben 9
2.4. Manuális játék és csúcsra kattintás 11
2.5. Keresőalgoritmusok beállítása és alkalmazása 12
2.6. Algoritmus lépéseinek vezérlése 13
2.7. Állapotgráf színkódolás szemléltetése 15

3.1. Az állapot ábrázolás osztálydiagramja 21
3.2. A keresőalgoritmusok osztálydiagramja 22
3.3. Az játéktábla megjelenítéséhez használt osztályok osztálydiagramja . 23
3.4. Az állapotgráf megjelenítéséhez használt osztályok osztálydiagramja, a nyolcasfa megvalósítása a projekthez szabva 24
3.5. Egységtesztek sikeres futtatása a szerkesztőben 33

Táblázatok jegyzéke

2.1. Támogatott platformok és futtatási utasításaik 7
2.2. A 3D-s állapotgráf csúcsainak és éleinek színmagyarázata 14

3.1. Felhasználói történetek 20
3.2. Egységtesztek a programhoz 32
3.3. Felhasználói történeteken alapuló tesztesetek 35